FINAL COURSE PROJECT REPORT

Credence AI

AI Based Digital Certificate Tamper Detection and Verification System

Artificial Intelligence Course Project

CSE311

Submitted by

Pranathi TR - 2024BCS0069

Rupasri M - 2024BCS0077

Peswika Bavagni - 2024BCS0257

Challa vidya - 2024BCS0157

Semester 5, Batch 2

Submitted to

Prajeesh Sir

Indian Institute of Information Technology Kottayam

October 2026

https://github.com/ptr25/credence-ai

1 Project overview

Abstract

Credence AI checks whether an uploaded certificate matches a trusted registered original. It combines visual comparison with a trained classifier to distinguish harmless file changes from suspected content edits. The application also supports certificate issuance, a searchable registry and revocation.

The prototype achieved 100% accuracy on 90 held-out synthetic test pairs. This is a result on a small, controlled dataset; performance on real certificates and unfamiliar layouts has not been established.

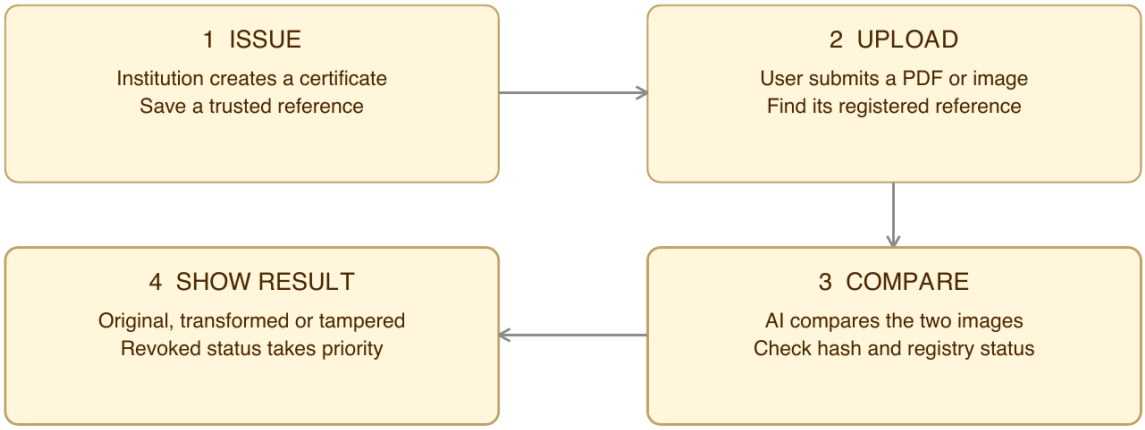


Figure 1 The project in four steps

Why visual comparison is useful

Compression can change a file without changing its meaning. Editing a grade also changes the file. A hash detects that bytes changed; the visual model helps distinguish these two situations.

Reading guide

Architecture: page 3 | AI model: page 4 | Dataset: page 5
Results: page 6 | Interface: page 7 | Demonstration: page 8
Conclusion and references: page 9
Source code and repository: page 10

2 Frontend backend and storage

The frontend is what the user sees. The backend receives requests, runs the checks and returns the result. The models run in Python on the backend.

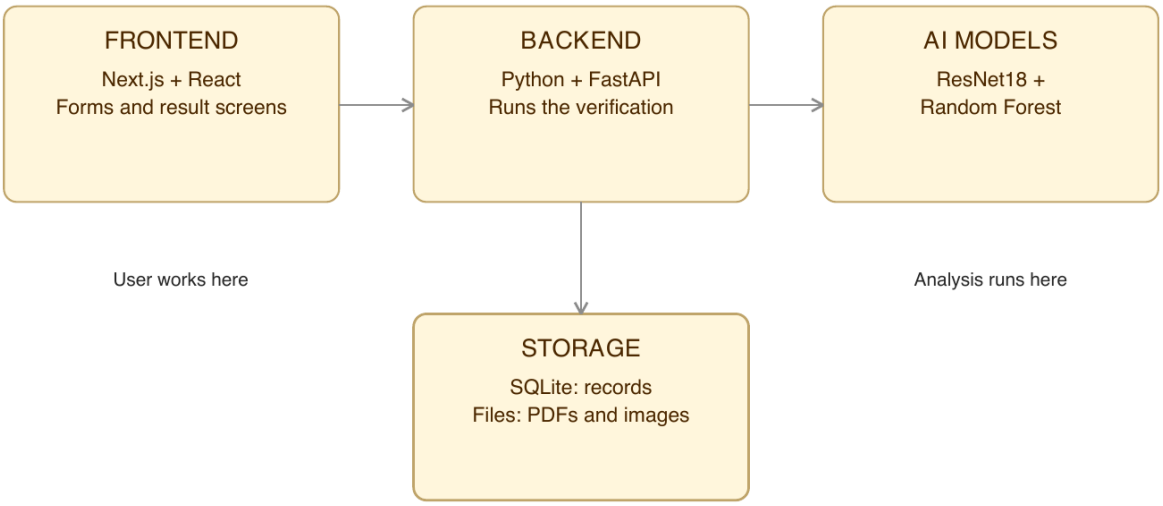


Figure 2 Four parts of the application

Part	Main responsibility
Frontend	Next.js, React and TypeScript display forms, uploads, registry and results.
Backend	FastAPI handles login, issuance, verification and revocation.
AI	PyTorch runs ResNet18; scikit-learn runs the trained Random Forest.
Storage	SQLite stores certificate details and verification history. Local files store PDFs, images and difference maps.

One request from start to finish

The user uploads a certificate → FastAPI finds the original → the models compare both images → the backend saves the analysis → the frontend displays the result.

Key backend checks

The backend uses SHA-256 for exact-file matching and checks whether the certificate has been revoked. A QR link opens the verification portal. A hidden-ID prototype helps identify a document, but does not prove that its content is unchanged.

3 How the AI works

There are two model roles: ResNet18 extracts visual features; Random Forest learns how to classify the comparison.

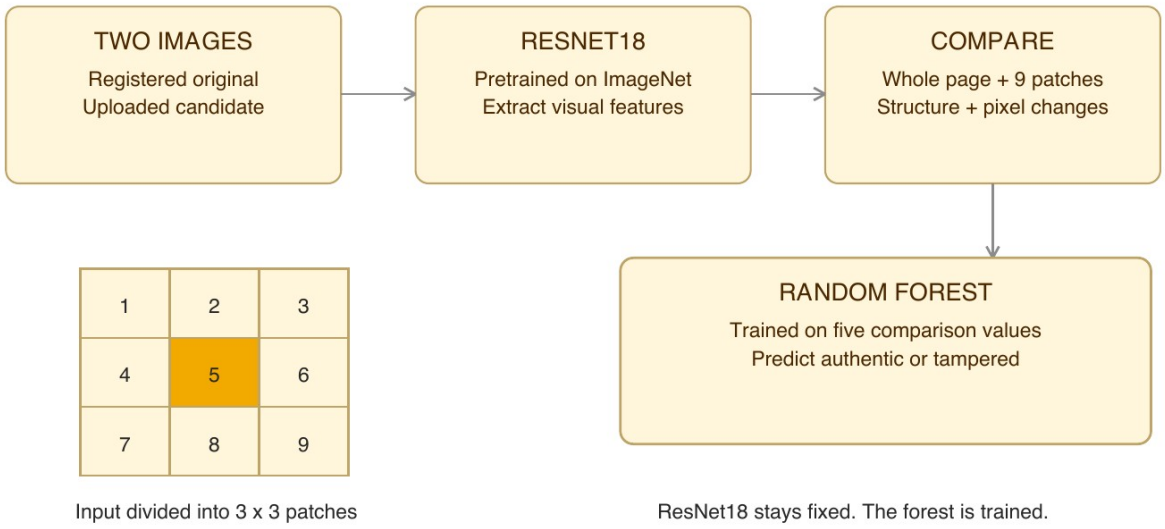


Figure 3 From two certificate images to a prediction

Step	Simple explanation
Prepare	Convert each document to a 900 × 636 image. ResNet18 receives 224 × 224 crops using its standard transform.
Extract	ResNet18 produces 512 numbers for the whole page and for each of nine patches. These numbers describe visual patterns.
Compare	Build five values: whole-page similarity, average patch similarity, lowest patch similarity, SSIM and mean pixel difference.
Predict	A Random Forest with 150 trees combines the five values and predicts authentic or tampered.

SSIM measures image structure. Pixel difference measures how much the corresponding pixels changed. Patches help expose a small local edit that can be hidden by high whole-page similarity.

ResNet18 uses existing ImageNet weights and is not fine-tuned. The Random Forest is the component trained for this project.

4 Training data and model selection

The project generates its own labeled data. It does not use a collected dataset of real institutional certificates.

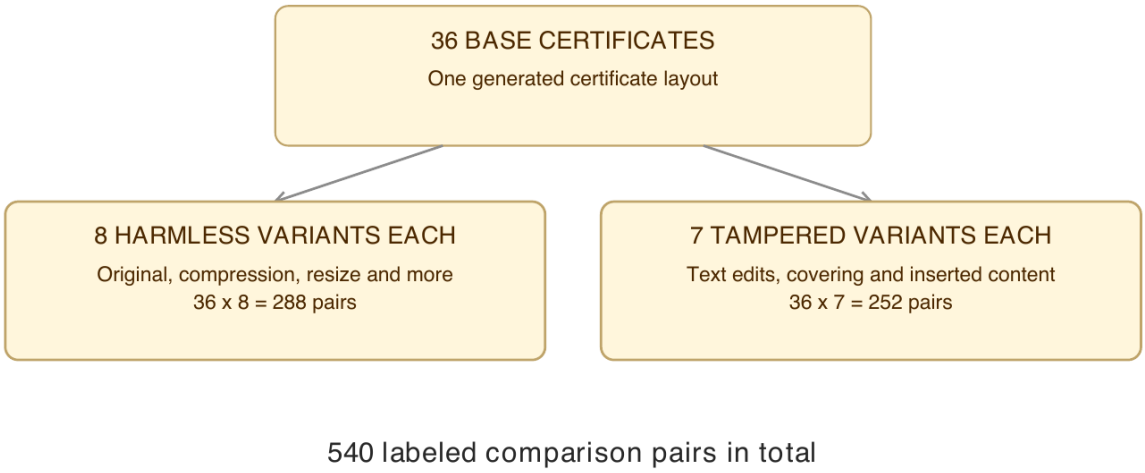


Figure 4 How the synthetic dataset is created

Harmless changes include JPEG compression, resizing, blur, brightness, contrast and noise. Tampered examples include grade, name and ID edits, plus covered, inserted or removed content.

Split	Base certificates	Comparison pairs	Purpose
Training	24	360	Learn patterns
Validation	6	90	Choose the model
Test	6	90	Measure final results

All variants of one base stay in the same split. This prevents the same original from appearing in both training and testing. However, all splits still share one layout and the same kinds of generated edits.

Why Random Forest was selected

Model	Validation F1
Logistic Regression baseline	0.9231
Random Forest	1.0000

The higher validation F1 selected Random Forest. The held-out test set was then used for the final evaluation. Issuing a new certificate does not retrain the models.

5 Results and confusion matrix

The test set contains 90 pairs: 48 authentic or harmlessly transformed, and 42 tampered. The model classified all 90 correctly.

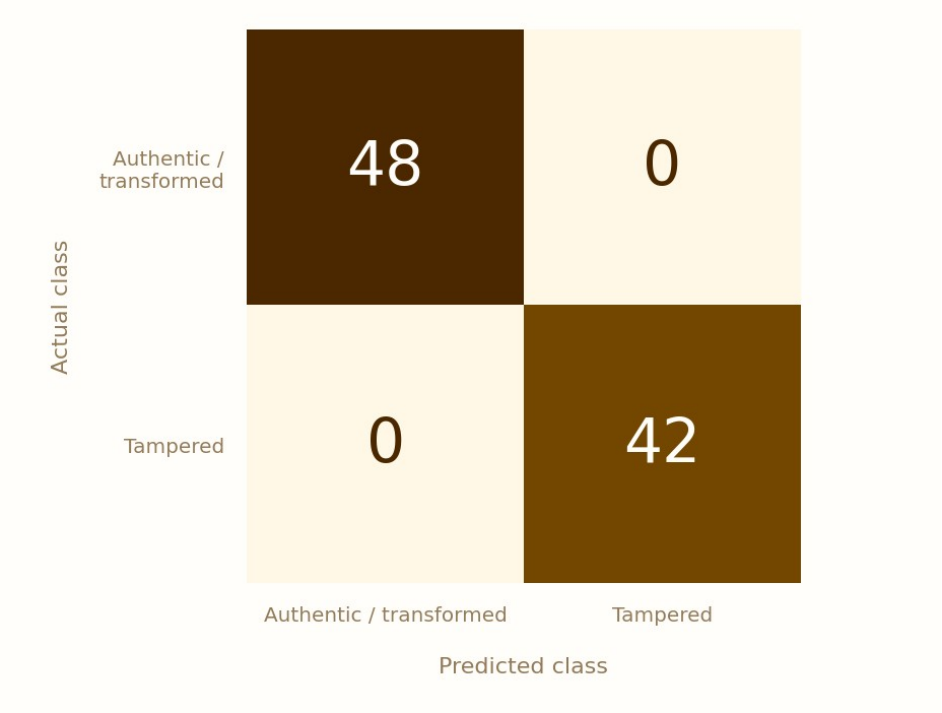


Figure 5 Actual classes are rows and predicted classes are columns

What the matrix means	Count
Authentic correctly accepted	48
Tampered correctly detected	42
Harmless files incorrectly flagged	0
Tampered files incorrectly accepted	0

Accuracy 100% | Precision 100% | Recall 100%
F1 1.00 | ROC AUC 1.00

Whole-page similarity alone achieved 81.11% accuracy. Adding patch comparisons reached 100%. Adding SSIM and pixel difference did not improve accuracy further on this test set.

These scores apply to this synthetic test only. Six held-out base certificates are too few to establish real-world reliability.

6 User interface

The interface brings certificate issuance, the registry, verification and evaluation into one workspace. Its cream, amber and brown palette keeps the screens consistent.

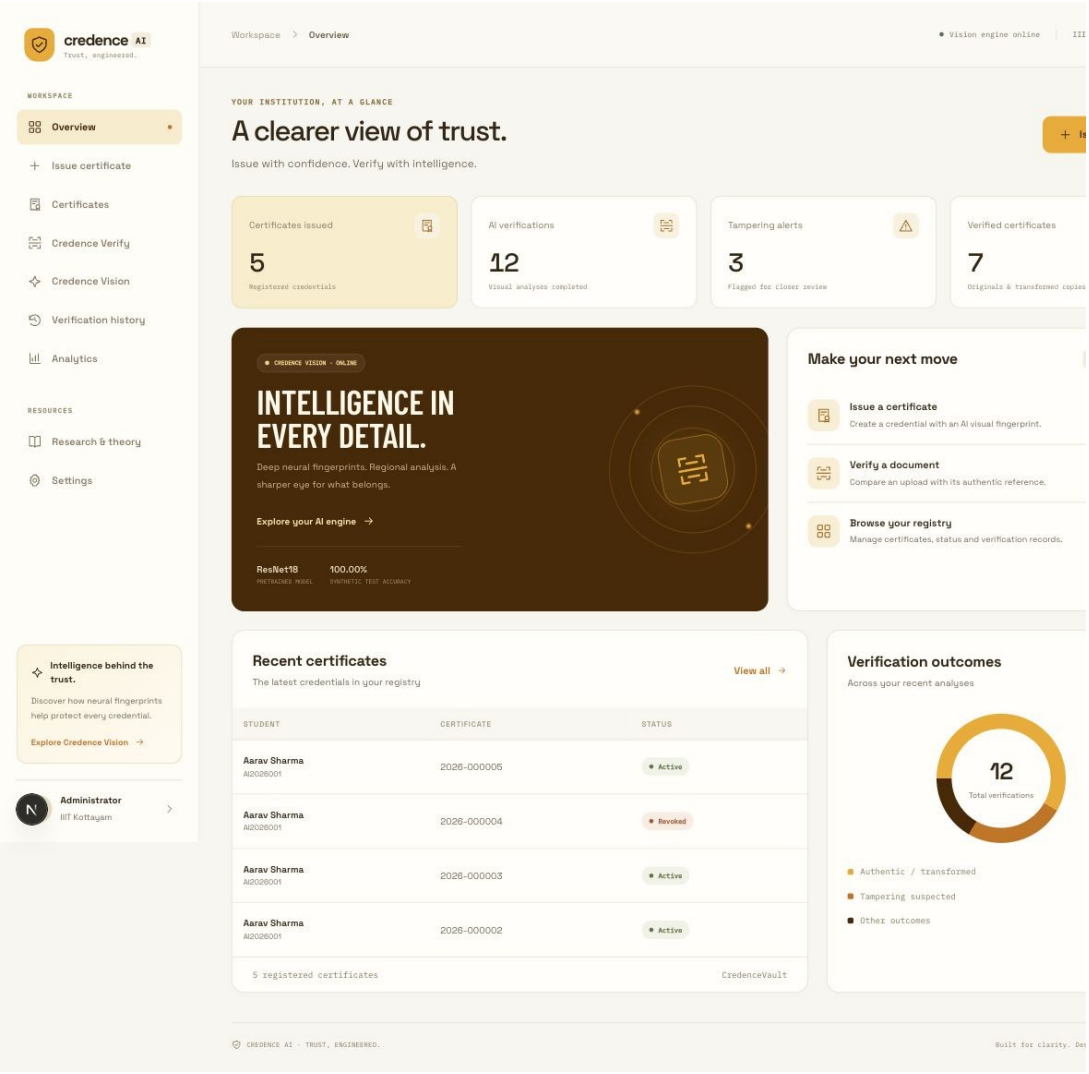


Figure 6 Captured institution dashboard from the implemented application

Screen	What the user does
Issue certificate	Enter student details and generate a registered PDF.
Registry	Find, download or revoke an issued certificate.
Verify	Upload a PDF or image and read the analysis.
Credence Vision	Inspect the model information and evaluation results.

Dashboard counts show application activity. They are separate from the 540 training and evaluation pairs.

7 Verification example

The saved demonstration compared an original PDF, a compressed copy and a grade-edited copy of the same registered certificate.

Uploaded file	Observed result
Original PDF	VERIFIED
Compressed copy	TRANSFORMED COPY
Grade-edited copy	TAMPERING SUSPECTED

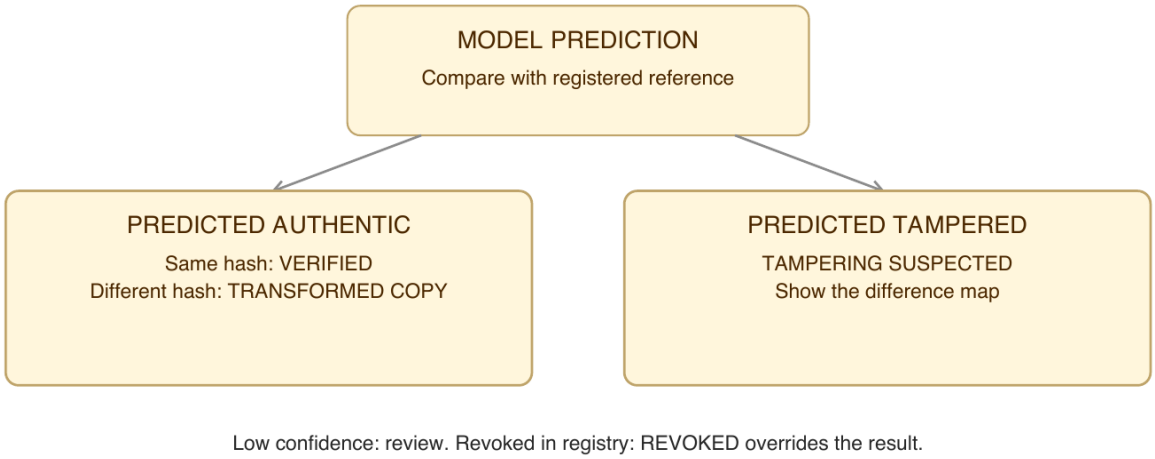


Figure 7 How the application combines prediction and registry checks



Figure 8 The red difference overlay highlights the edited grade region

The overlay shows changed pixels, not neural attention. A local edit can leave most of the page looking similar, which is why the model compares several signals together.

8 Conclusion and references

What the project demonstrates

Credence AI connects a working web interface, a Python backend, a pretrained vision model and a trained classifier. In the controlled demonstration, it distinguishes an unchanged certificate, a harmlessly compressed copy and an edited grade.

Testing and limitations

The recorded development checks passed the integration test and the frontend production build. The integration test covers issuance, verification, transformed and tampered examples, and revocation.

- The dataset uses 36 generated originals and one layout. Real scans and unfamiliar templates need separate testing.
- The system needs a trusted registered reference. It does not establish whether an unknown issuer is legitimate.
- There is no OCR-based checking of names or grades. Image alignment and the hidden-ID prototype have limitations.

Future improvements

Use more real document layouts, test realistic edits and scans, align images before comparison, and add OCR for text fields. Stronger authentication and privacy controls are needed before deployment.

References

[1] He et al. Deep Residual Learning for Image Recognition. CVPR, 2016. arxiv.org/abs/1512.03385

[2] Breiman. Random Forests. Machine Learning 45, 5–32, 2001. doi:10.1023/A:1010933404324

[3] Wang et al. Image Quality Assessment: From Error Visibility to Structural Similarity. IEEE TIP 13(4), 600–612, 2004.

[4] Official documentation: PyTorch torchvision ResNet18 and scikit-learn RandomForestClassifier.

Implementation evidence and acknowledgement

Results come from the saved metrics, dataset metadata and verification records. Screenshots show the implemented application. AI-assisted tools supported coding, debugging, interface development and documentation.

9 Key code and repository

Browse source code and reports on GitHub

Selected Python excerpts, formatted for readability. Variables are defined in the linked source files; these are not standalone programs.

Pretrained visual feature extractor

backend/app/services/vision_service.py

```
weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
model.fc = torch.nn.Identity()
model.eval()
```

The final classification layer is removed so ResNet18 returns visual features.

Keep related samples in one split

backend/app/ai/train_classifier.py

```
ids = np.random.default_rng(42).permutation(36)
tr = np.isin(groups, ids[:24])
va = np.isin(groups, ids[24:30])
te = np.isin(groups, ids[30:])
```

The split is by base certificate, not by individual transformed image.

Fit and select the classifier

backend/app/ai/train_classifier.py

```
for name, m in candidates.items():
    m.fit(x[tr], y[tr])
    s = f1_score(y[va], m.predict(x[va]))
    if s > bestscore:
        best, bestscore = (name, m), s
```

Logistic Regression and Random Forest are fitted; validation F1 chooses the winner.

Combine AI with integrity and revocation

backend/app/main.py

```
match = hashlib.sha256(data).hexdigest() == obj['documentHash']
result = ('TAMPERING_SUSPECTED' if pred else
          ('VERIFIED' if match else 'TRANSFORMED_COPY'))
if confidence < .65: result = 'REVIEW_RECOMMENDED'
if obj['status'] == 'REVOKED': result = 'REVOKED'
```

Prediction supplies the class; file hash, confidence and registry status determine the result.